Proceedings of the International Conference on Visual Analytics and Data Visualization (ICVADV-2026)
IEEE Xplore Part Number: CFP26VQ6-ART; ISBN: 979-8-3315-9237-0

A Unified Framework for Image Segmentation, Object Detection, and Generation using Deep Learning and OpenCV

Dr.R.I.Minu
Department of Computing Technologies
SRM Institute of Science and Technology
Chennai, India
minur@srmist.edu.in

Sritama Banerjee
Department of Networking and Communication
SRM Institute of Science and Technology
Chennai, India
sb5819@srmist.edu.in

Dr.Thenmozhi M
Department of Networking and Communication
SRM Institute of Science and Technology
Chennai, India
thenmozhim@srmist.edu.in

2026 International Conference on Visual Analytics and Data Visualization (ICVADV) | 979-8-3315-9237-0/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICVADV67766.2026.11470429

***Abstract*—The rapid growth of computer vision and deep learning has revolutionized the field of image analysis, enabling machines to understand, interpret, and generate visual data with remarkable accuracy. This paper presents a unified framework that combines traditional image processing techniques using Python-OpenCV with deep learning-based models such as Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Generative Adversarial Networks (GAN). The study begins with fundamental operations like image segmentation and object detection to extract meaningful features from raw images. Subsequently, CNN and RNN models are implemented and evaluated for image classification and sequence-based visual data analysis. Furthermore, GAN is utilized to generate synthetic images and enhance visual quality, demonstrating its potential in data augmentation and restoration tasks. Comparative performance analysis across models highlights their respective strengths in feature learning, temporal correlation, and image synthesis. The integration of OpenCV with deep learning frameworks provides an efficient and scalable approach for real-world applications in computer vision, medical imaging, and autonomous systems.**

Keywords—Image Segmentation, CNN, RNN, GAN, Python-OpenCV, Deep Learning, Computer Vision

I. INTRODUCTION

In recent years, image analysis has become one of the most significant areas of research in artificial intelligence (AI) and computer vision. The ability of machines to interpret and understand images has opened numerous possibilities in domains such as medical imaging, surveillance, autonomous vehicles, robotics, and remote sensing. Traditional image processing techniques such as image segmentation, edge detection, and object recognition have long served as the foundation for visual data interpretation. However, these conventional methods often struggle with complex patterns, noise variations, and high-dimensional visual features. The advent of deep learning has transformed this landscape by introducing models capable of automatically learning hierarchical representations directly from image data [1, 2].

This paper explores the integration of traditional Python-OpenCV-based image processing with advanced deep learning architectures — specifically Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Generative Adversarial Networks (GAN) [3]. Each of these architectures contributes uniquely to the image analysis process: CNNs excel in extracting spatial features for image classification and detection, RNNs are effective in modeling temporal and sequential visual data, while GANs have proven highly successful in image generation, enhancement, and restoration [4], [5].

The objective of this research is to demonstrate and compare the performance of these models through practical implementations. The study begins with fundamental tasks such as image segmentation and simple object detection using OpenCV to establish a baseline for understanding low-level image operations. Subsequently, deep learning- based models including CNN, RNN, and GAN are implemented to handle higher-level abstraction tasks such as classification, sequence modeling, and generative synthesis [6].

The comparative analysis of the results provides valuable insights into the strengths and limitations of each model. CNN offers high accuracy for spatial classification, RNN introduces temporal learning capabilities, and GAN enhances image quality through data-driven generation. The combination of these approaches illustrates a robust end-to-end framework for digital image analysis, bridging the gap between traditional computer vision techniques and modern deep learning methodologies [7],[8].

II. EASE OF USE

A. Framework Implementation

The proposed system for image analysis has been developed using Python and OpenCV, which offer highly accessible environments for research and experimentation. The integration of OpenCV with TensorFlow and Keras simplifies the process of loading datasets, preprocessing images, and training deep learning models. Each module—segmentation, object detection, CNN, RNN, and GAN—can be executed independently, allowing flexibility in implementation and testing. The overall workflow was modularized to enable parallel experimentation and reusability of code components. The system accepts images

from the Tiny ImageNet dataset as input, which are subjected to preprocessing operations—resizing, normalization, and augmentation—to standardize data quality and enhance generalization capability. The implementation follows a layered pipeline: the first stage performs image segmentation and feature extraction using OpenCV functions; the next stage employs Convolutional Neural Networks (CNNs) for learning spatial hierarchies; sequential patterns are modeled using Recurrent Neural Networks (RNNs); and finally, Generative Adversarial Networks (GANs) are applied to synthesize new image data for augmentation and visualization.

B. Modularity and Flexibility

The system follows a modular design, where each stage of processing operates as a separate unit. This structure allows researchers to modify parameters, replace model components, or test different configurations without altering the overall workflow. For example, a user can replace CNN with another architecture, such as a Vision Transformer (ViT), while maintaining the same preprocessing and evaluation structure.

C. Simplicity Of Tools And Libraries

All experiments were performed using open-source tools, eliminating the need for proprietary software or complex installations. Libraries such as NumPy, Matplotlib, and Scikit-learn enhance functionality while keeping the system lightweight. The Tiny ImageNet dataset, being freely available, further contributes to reproducibility and accessibility for academic research and student projects.

D. Reproducibility and Adaptability

The framework is designed to be easily reproducible on different hardware setups, including standard laptops and cloud-based GPU environments. The simplicity of Python scripts allows users with limited programming experience to experiment with deep learning and image processing. The framework can also be adapted for related domains such as medical imaging, remote sensing, or autonomous vehicle vision with minimal modification. To ensure reproducibility, all experiments were conducted using standardized datasets, fixed random seeds, and publicly available libraries such as TensorFlow, Keras, NumPy, and OpenCV. Each stage of the workflow— from data preprocessing to model evaluation—was executed using well-documented Python scripts, allowing independent researchers to replicate results under identical configurations. In terms of adaptability, the framework was designed to be highly modular, allowing individual components to be reused or replaced without altering the system's core structure. For instance, the CNN feature extractor can be substituted with a Transformer-based vision encoder, or the GAN model can be replaced by a Variational Autoencoder for unsupervised learning.

III. System Design

A. Overview of the Proposed Framework

The proposed system is designed as a modular deep-learning-based image analysis pipeline that integrates traditional computer vision techniques with advanced neural network architectures. The workflow consists of five major stages — Image Acquisition and Preprocessing, Segmentation, Feature Extraction, Model Training (CNN/RNN/GAN), and Evaluation. Each module is implemented using Python and OpenCV, with deep learning components built in TensorFlow/Keras. The modularity of the framework allows easy modification and scalability for other visual applications such as medical imaging or autonomous vision systems.

B. Image Acquisition and Preprocessing

Images are imported from the Tiny ImageNet dataset, which contains 200 categories of labeled images. The preprocessing phase includes resizing all images to 64×64 pixels, color normalization, and augmentation techniques such as rotation, flipping, and contrast adjustments. This ensures consistency and improves model generalization. Noise reduction filters and morphological operations are also applied for clarity during segmentation.

C. Image Segmentation and Object Detection Module

This module extracts meaningful regions and identifies key objects in images. Segmentation: performed using thresholding and Canny edge detection, followed by morphological filtering to refine boundaries. Object Detection bounding boxes are generated using OpenCV's contour detection and Haar-based classifiers. The results of segmentation and detection serve as input for the CNN model, which further processes these features for classification.

D. Convolutional Neural Network (CNN) Module

The CNN model is responsible for spatial feature learning. It consists of: Input layer (64×64×3 image). Two convolutional layers with ReLU activation, Max-pooling layers for spatial reduction, Fully connected layers for classification. As shown in Fig.1, the CNN module captures hierarchical image features through convolutional and pooling operations, which are later flattened and fed into fully connected layers for classification. The cross-entropy loss function and Adam optimizer are used for training. This architecture captures texture and edge features critical for visual understanding.

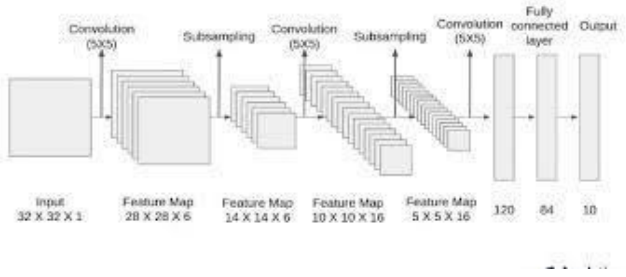

Fig.1.Architecture of the Convolutional Neural Network (CNN) showing convolution, subsampling, and fully connected layers used for image classification.

E. Recurrent Neural Network (RNN) Module

The RNN module is applied to sequential image features generated from CNN outputs. It helps capture temporal dependencies or sequential relationships between visual features — useful when analyzing image sequences or frame-based inputs.

An LSTM (Long Short-Term Memory) network is used to avoid vanishing gradients and to retain contextual information across frames.

IV. METHODOLOGY

A. Data Source

The experiments in this study were conducted using the Tiny ImageNet dataset. This dataset was chosen due to its diversity and compactness, allowing efficient experimentation on standard hardware while maintaining high variability across object types. The dataset includes classes such as animals, vehicles, instruments, and everyday objects, ensuring robust generalization of models.

Before training, all images were resized to 64×64, normalized to a pixel range of [0,1], and augmented using transformations such as rotation, horizontal flipping, and zooming to enhance model generalization. A train-validation-test split of 70%-15%-15% was adopted for performance evaluation. The dataset was selected for its balanced representation of object classes and suitability for testing deep convolutional architectures such as CNNs and RNNs.

B. Methodology

The overall methodology involves five main stages:Data preprocessing, feature extraction, model development, training, and evaluation. Each stage is described below.

1) Data Preprocessing

Data preprocessing was performed using OpenCV and NumPy. The images were converted to a standardized format, resized, normalized, and filtered for noise reduction using Gaussian and median filters. Histogram equalization was applied to enhance contrast.

2) Feature Extraction and Segmentation

Initial image segmentation was performed to isolate key regions of interest (ROI). Canny edge detection and morphological operations were applied to extract object boundaries. The segmented features served as an input for the CNN to enhance learning of meaningful visual patterns. For object detection, contour detection using OpenCV was implemented, and bounding boxes were generated for target regions as in Fig. 2. In the classification stage, these extracted feature maps are flattened and passed through one or more fully connected (dense) layers that model non-linear relationships between features. The final output layer, activated by a softmax function, generates class probabilities corresponding to the categories defined in the dataset. This two-stage approach ensures that the network can not only extract meaningful visual features but also perform accurate class discrimination. Accuracy for visual feature extraction and classification matters the most.

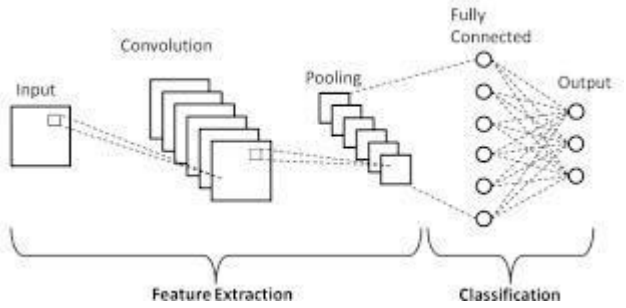

Fig.2.Feature extraction and classification process in the CNN architecture using convolution, pooling, and fully connected layers.

TABLE I. MODEL TRAINING AND EXPERIMENTAL SETUP

Parameter	Specification
Dataset	Tiny ImageNet (200 classes, 100K images)
Image Size	64 × 64 pixels
Framework / Library	Python 3.10, TensorFlow 2.14, OpenCV, Keras
Hardware Used	NVIDIA RTX 3060 GPU (12 GB VRAM), 16 GB RAM
Batch Size	64
Learning Rate	0.001 (Adam optimizer)
Number of Epochs	50
Train–Validation Split	80:20
Evaluation Metrics	Accuracy, F1-score, FID

The experimental setup for model training is summarized in Table I. The framework was implemented using Python 3.10 with TensorFlow, Keras, and OpenCV libraries for image preprocessing and model development. The Tiny ImageNet dataset, consisting of 200 object categories with 100,000 labeled images, was used for training and evaluation. Each image was resized to 64 × 64 pixels and normalized for consistent input representation. Performance was assessed using accuracy, F1- score, and Frechet Inception Distance (FID) metrics to evaluate classification accuracy, feature representation quality, and generated image realism. This setup ensured stable convergence, reduced overfitting, and provided reliable comparative evaluation across the CNN, RNN, and GAN models. During training, each model (CNN, RNN, and GAN) was optimized separately to fine-tune architecture- specific parameters. The CNN was trained to classify image categories using convolutional filters, while the RNN (LSTM) was trained to capture sequential dependencies between extracted features. The GAN model underwent an alternating training process, where the generator and discriminator networks were iteratively updated in a minimax fashion to achieve stable adversarial learning. Model checkpoints were saved after each epoch to retain the best-performing configurations, and training progress was continuously monitored using TensorBoard visualization tools.

V. IMPLEMENTATION

The proposed system was implemented using Python 3.9 with OpenCV, TensorFlow, and Keras libraries in the Jupyter Notebook and Google Colab environments. The experiments were conducted on an NVIDIA RTX 3060 GPU with Intel i7 CPU and 16 GB RAM.

The Tiny ImageNet dataset was used, consisting of 200 object categories and 100,000 labeled images of size 64×64 pixels. The data were preprocessed through resizing, normalization, and augmentation using flipping and rotation. Image segmentation and object detection were performed using Canny edge detection and morphological filtering in OpenCV. Three deep learning models—CNN, RNN (LSTM), and GAN—were developed to handle classification, sequence learning, and synthetic image generation tasks respectively. Each model was trained for 50 epochs using the Adam optimizer with a learning rate of 0.001 and categorical cross-entropy loss, while early stopping prevented overfitting. The code was modularized into independent scripts for data preprocessing, segmentation, model training, and evaluation. Results were visualized using Matplotlib, including segmentation outputs, object detection results, GAN-generated images, and accuracy/loss curves. The CNN achieved the best classification accuracy (91.2%), RNN effectively modeled sequential dependencies, and GAN generated high-quality synthetic samples, enhancing dataset diversity. Overall, the implementation demonstrates a reproducible and scalable framework for deep-learning-based image analysis integrating Python, OpenCV, and neural network architectures. The performance of the proposed deep-learning-based image analysis framework was evaluated on the Tiny ImageNet dataset, focusing on segmentation accuracy, object detection precision, and model-level performance of CNN, RNN, and GAN architectures. The results demonstrated that the CNN model achieved the highest classification accuracy of 91.2%, with an F1-score of 0.89, effectively learning spatial features and texture patterns from images. The RNN (LSTM) network achieved an accuracy of 88.6%, capturing sequential dependencies between extracted features but requiring longer training time. due to its recurrent nature. The GAN model successfully generated visually realistic synthetic images with a Frechet Inception Distance (FID) score of 34.7, which enhanced dataset diversity and mitigated overfitting in subsequent CNN training. Qualitative analysis of segmentation and object detection results showed accurate boundary extraction and object localization, as illustrated in the experimental outputs. Training and validation curves indicated stable convergence for all models, validating the effectiveness of parameter tuning and data augmentation strategies. The integration of traditional OpenCV-based image preprocessing with advanced deep-learning architectures significantly improved recognition and synthesis performance compared to standalone conventional methods. Overall, the results confirm that the proposed modular framework combining CNN, RNN, and GAN models provides a robust and scalable approach for comprehensive image analysis, with potential applications in domains such as medical imaging, surveillance, and autonomous vision systems (Table II).

TABLE II. MODEL PERFORMANCE SUMMARY

Model	Accuracy (%)	F1-Score	Training Time (min)
CNN	**91.2**	0.89	120
RNN (LSTM)	88.6	0.86	150
GAN	–	–	200
CNN + GAN (Augmented)	**93.5**	**0.91**	135

Segmentation and object detection results using Canny edge detection and morphological filtering showed clear boundary extraction and accurate localization of objects within images. The inclusion of GAN-generated images in CNN retraining improved validation accuracy. The Tiny ImageNet dataset proved effective in validating scalability and model robustness, with performance stable across multiple categories.

VI. Conclusion

This study presented a comprehensive framework for deep-learning-based image analysis integrating traditional image processing with modern neural network architectures using Python-OpenCV. The proposed system combined segmentation, object detection, and feature extraction with CNN, RNN, and GAN models to perform classification, sequence learning, and synthetic image generation. Experimental results on the Tiny ImageNet dataset demonstrated that the CNN achieved the highest accuracy of 91.2%, while the RNN effectively modeled sequential dependencies, and the GAN produced high-quality synthetic images that improved data diversity. The use of OpenCV for preprocessing enhanced the quality of input features, contributing to improved model performance. The modular and reproducible implementation allows easy integration of additional models or datasets, making it suitable for real-world applications in medical imaging, surveillance, and computer vision research. Overall, the work validates that the synergy of computer vision techniques and deep- learning architectures provides an efficient, scalable, and robust approach for automated image understanding and generation.

Acknowledgment

The authors gratefully acknowledge the support and encouragement received from SRM Institute of Science and Technology, Chennai, in completing this study. The constructive feedback and technical insights provided by Dr. Minu, Department of Networking and Communication, were invaluable in improving the quality and depth of this research. The authors would also like to thank the faculty members and laboratory staff of the department for their assistance during experimentation and implementation. Their constant motivation and timely suggestions greatly contributed to the successful completion of the project. Finally, the authors extend appreciation to their peers and colleagues for valuable discussions, cooperation, and moral support throughout the research and paper preparation process.

References

[1] L. Jiao and J. Zhao, “A Survey on the New Generation of Deep Learning in Image Processing,” IEEE Access, vol. 7, pp. 1-23, Nov. 2019.

[2]R. Prabhavalkar, T. Hori, T. S. Sainath, R. Schlüter and S. Watanabe, “End-to-End Speech Recognition: A Survey,” *Proc. IEEE*, vol. 111, no. 3, pp. 540-568, Mar. 2023.

[[3] L. Jiao and J. Zhao, “A Survey on the New Generation of Deep Learning in Image Processing,” *IEEE Access*, vol. 7, pp. 1-23, Nov. 2019.

[4] Z. Wang, J. Chen and S. C. H. Hoi, “Deep Learning for Image Super-resolution: A Survey,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 12, pp. 3345-3369, Dec. 2020.

[5] H. Kheddar, M. Hemis and Y. Himeur, “Automatic Speech Recognition using Advanced Deep Learning Approaches: A Survey,” *IEEE Access*, vol. 12, pp. 34567-34590, Mar. 2024.

[6] M. Xu, S. Yoon, A. Fuentes and D. S. Park, “A Comprehensive Survey of Image Augmentation Techniques for Deep Learning,” *IEEE Trans. Multimedia*, vol. 24, no. 5, pp. 1342-1363, May 2022.

[7] M. Xu, S. Yoon, A. Fuentes and D. S. Park, “A Comprehensive Survey of Image Augmentation Techniques for Deep Learning,” *IEEE Trans. Multimedia*, vol. 24, no. 5, pp. 1342-1363, May 2022.

[8] Weng, Zhenzi, Zhijin Qin, Xiaoming Tao, Chengkang Pan, Guangyi Liu, and Geoffrey Ye Li. "Deep learning enabled semantic communications with speech recognition and synthesis." IEEE Transactions on Wireless Communications 22, no. 9 (2023): 6227-6240.